

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Automatic Attribution Graphs Summarization for Circuit Tracing Pipeline

Phạm Anh Tú

tu.pa225463@sis.hust.edu.vn

Program: Data Science and Artificial Intelligence

Supervisor: Dr. Nguyễn Đức Anh

Department: Computer Science

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

Automatic Attribution Graphs Summarization for Circuit Tracing Pipeline

Phạm Anh Tú

tu.pa225463@sis.hust.edu.vn

Program: Data Science and Artificial Intelligence

Supervisor: Dr. Nguyễn Đức Anh _____

Signature

Department: Computer Science

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENT

First and foremost, I would like to express my deepest gratitude to my advisor, Dr. Nguyen Duc Anh. His invaluable guidance, mentorship, and insightful feedback were instrumental in bringing this research to fruition. His constructive critiques significantly shaped the direction of this thesis, and his steadfast encouragement has profoundly helped me grow as a researcher.

Secondly, I am endlessly grateful to my family and friends for their unconditional support and sacrifices. Without their love and belief in me, I would not be the person I am today.

Finally, I would like to extend my sincere thanks to HUST and SOICT. Over the past four years, my time here has taught me more than I ever have in my life. Beyond the academic knowledge, the time I had in this institution has shaped my understanding of what it truly means to be human: our limitations, our responsibilities, and the prestige of perseverance.

Thank you all.

ABSTRACT

Large language models have become effective systems for text generation and reasoning, but their internal computations remain difficult to inspect. This opacity limits the ability of researchers to explain why a model produces a prediction, diagnose failures, and build reliable evidence about model behavior. Mechanistic interpretability addresses this problem by reverse-engineering model computations into human-understandable mechanisms. Within that field, circuit tracing studies the specific internal pathways that support one behavior or one prediction. Recent circuit-tracing methods use replacement models based on transcoders to express a model’s computation as an attribution graph, whose nodes are interpretable features and whose signed edges record linear effects leading to a target token.

Attribution graphs provide a faithful and prompt-specific view of model computation, but they are still too large for direct human reading. A single prompt can produce thousands of feature nodes and many weighted edges, so existing workflows rely on manual pruning and manual grouping by expert inspection. This thesis chooses automated attribution-graph summarization as the central approach because it preserves the mechanistic object produced by circuit tracing while replacing the most time-consuming human steps with explicit objectives. The proposed method takes an attribution graph as input and returns a compact summary graph. It first prunes nodes and edges using influence and relevance scores, then clusters surviving feature nodes into supernodes under an objective that rewards atomicity and causal preservation, and finally labels and assembles the resulting directed acyclic summary graph for inspection.

The main contributions of this thesis are a formalization of faithful attribution-graph summarization, an automated three-stage pipeline for pruning, clustering, and visualizing such graphs, and an empirical evaluation on analogy and multi-hop reasoning prompts. The results show that the summary graphs are interpretable and faithful to the original attribution graphs.

Student

(Signature and full name)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Problem Statement.....	1
1.2 Background and Problems of Research	1
1.3 Research Objectives and Conceptual Framework	2
1.4 Contributions	2
1.5 Organization of Thesis	2
CHAPTER 2. LITERATURE REVIEW	4
2.1 Scope of Research	4
2.2 Transformer	4
2.3 Mechanistic Interpretability and Circuits.....	5
2.4 Sparse Autoencoders and Transcoders	6
2.5 Circuit Tracing Backbone.....	7
2.5.1 Replacement Model	7
2.5.2 Attribution Graphs	8
2.5.3 Interventions.....	10
2.6 Automated Interpretation Methods	10
2.7 Related Works	11
CHAPTER 3. METHODOLOGY	13
3.1 Overview	13
3.2 Problem Formulation	13
3.2.1 Input	13
3.2.2 Output.....	14
3.2.3 Desiderata of the summary graph	14

3.3 Method	15
3.3.1 Pruning	15
3.3.2 Clustering.....	17
3.3.3 Visualizing	20
3.4 Chapter Summary	22
CHAPTER 4. NUMERICAL RESULTS.....	23
4.1 Overview	23
4.2 Common Experimental Setup	23
4.2.1 Datasets	23
4.2.2 Environment and implementation	24
4.2.3 Default pipeline configuration.....	24
4.3 Experiment 1: Pruning Faithfulness.....	25
4.3.1 Question and comparison.....	25
4.3.2 Metrics	25
4.3.3 Results	26
4.4 Experiment 2: Causal Supernode Clustering	27
4.4.1 Question and setup	27
4.4.2 Baselines.....	27
4.4.3 Metrics	28
4.4.4 Results	29
4.5 Experiment 3: Intervention Validation via Entity-swap.....	30
4.5.1 Question and setup.....	30
4.5.2 Intervention protocol	30
4.5.3 Metrics	30
4.5.4 Results	31
4.6 Chapter Summary.....	32

CHAPTER 5. CONCLUSIONS	33
5.1 Summary	33
5.2 Future Work	34
REFERENCE	38
APPENDIX	40
A. APPENDIX.....	40
A.1 SHAP Value Calculation	40

LIST OF FIGURES

Figure 2.1	The decoder-only transformer as a residual network. Each sublayer reads the residual stream and adds its output back into it, so the final representation is a sum of contributions from the embedding, every attention head, and every MLP.	5
Figure 2.2	Dashboard visualization of an attribution graph. The interface shows the graph over prompt positions and layers, lists input and output features, and displays a selected subgraph used for circuit inspection.	9
Figure 3.1	Methodology pipeline.	14
Figure 4.1	Summary graph and intervention view for the prompt “The saying goes: fox is to cub as goat is to”. The model output changes after swapping Output features that promote “kid” with features that promote “calf”.	31

LIST OF TABLES

Table 4.1	Pruning results on 100 BATS analogy graphs at edge retained-mass threshold 0.95 and node retained-mass threshold 0.02. Values are means across graphs. Irrelevant feature and token counts are the numbers of retained feature and input-token nodes at irrelevant positions (lower is better); graph completeness and replacement score are the graph-faithfulness metrics of Eq. (4.2) and Eq. (4.1) (higher is better). The best value in each column is in bold and the second-best value is underlined; the $\alpha = 1.0$ baseline ignores token weights, so its normalization is not applicable.	27
Table 4.2	Clustering evaluation on entmax-pruned analogy and multihop graphs. Higher role gap indicates more coherent signed causal roles inside supernodes. Lower internalized edge mass and lower DAG loss indicate better preservation of visible causal structure. Gains are measured against the random same-size baseline. Best and <u>second-best</u> values are highlighted for each metric.	29
Table 4.3	Entity-swap supernode steering on 50 BATS analogy pairs per relation with the source factor fixed at -2 . Donor Output features are injected with the grouped donor factor. Hit rates are over all attempted pairs, and Δp values are steered probabilities minus clean probabilities. The best value for each relation and metric is shown in bold.	32

LIST OF ABBREVIATIONS

Abbreviation	Definition
BOS	Beginning-of-sequence token
CLT	Cross-layer transcoder; a sparse replacement module whose features can write to multiple later layers
DAG	Directed acyclic graph; the graph structure required for both attribution graphs and final summary graphs
ILP	Integer linear program; the exact clustering formulation used to assign feature nodes to supernodes
LLM	Large language model
MLP	Multilayer perceptron; the transformer sublayer replaced by transcoders in the circuit-tracing backbone
SAE	Sparse autoencoder; a dictionary-learning model that recovers sparse feature directions from activations
SHAP	SHapley Additive exPlanations; the input-token attribution method used as a relevance proxy

CHAPTER 1. INTRODUCTION

1.1 Problem Statement

Large language models achieve strong results across natural-language tasks, yet their internal computation remains largely opaque. Mechanistic interpretability seeks to reverse-engineer that computation into human-understandable algorithms rather than only relating inputs to outputs. A central goal of this effort is circuit tracing, which identifies the pathways of activation inside a model that implement a particular behaviour. Recent work makes this concrete: Ameisen et al. [1] use a cross-layer transcoder as a replacement model to extract an *attribution graph*, a directed graph whose nodes are interpretable features and whose edges are the linear effects between them, recording the steps the model takes to produce the next token for a given prompt. The attribution graph is a faithful, prompt-specific account of the model’s computation, and it is the object this thesis studies.

The attribution graph, however, is too large to read. A single prompt induces thousands of feature nodes joined by a dense web of edges, and making sense of one graph can take an expert hours, or becomes infeasible for long prompts. The graph must therefore be reduced to a compact form before a human can use it, but the reduction must not distort the computation it represents. This thesis addresses the resulting problem: how to summarize an attribution graph automatically into a small, human-readable graph that remains faithful to the model’s behaviour.

1.2 Background and Problems of Research

The circuit-tracing methodology already provides a way to shrink an attribution graph, but it relies on human judgment at every step. After building the graph, Ameisen et al. [1] prune it by keeping the nodes and edges that account for most of the influence on the output and discarding the rest, and then group the surviving features into *supernodes* by inspection, collecting features that play the same role into a single labelled unit so the circuit can be drawn as a small diagram. Both steps are manual: pruning thresholds are tuned by hand, and grouping a single circuit can take hours of expert effort, so the analysis does not scale beyond a handful of prompts.

Related efforts address other parts of the problem but leave the structural reduction open. Work on circuits in the neuron basis [2] avoids transcoder training and reconstruction error, yet the authors note that the resulting circuits are still too large to read and call for principled grouping methods. ADAG [3] addresses this by introducing attribution profiles to quantify the role of neurons in the graph and uses spectral

clustering to cluster them. This approach does not take structural information into account, making the clustering less mechanistically faithful. No existing method produces a compact and faithful structural summary of a signed attribution graph by jointly pruning and clustering its feature nodes under an explicit faithfulness objective.

1.3 Research Objectives and Conceptual Framework

This thesis aims to summarize attribution graphs automatically so that researchers can understand a model’s behaviour without manual reduction. The objective is a method that takes an attribution graph and returns a small *summary graph* of supernodes that is interpretable, faithful to the original computation, and produced without human intervention. The proposed solution responds directly to the limitations of the previous section. In place of hand-tuned pruning, it scores every node and edge by a combination of its downstream influence on the output and its upstream relevance to the input, and keeps only the important ones. In place of grouping features by inspection, it clusters them by a role similarity under an explicit objective that rewards atomic supernodes while preserving the causal edges between them. Finally, it aggregates the surviving edges into an acyclic, labelled supernode graph, so the output is a directed acyclic graph that a reader can follow at a glance. Each stage replaces one manual step with an automated, objective-driven procedure.

1.4 Contributions

This thesis makes three contributions:

- It formalizes the problem of summarizing an attribution graph, defining the summary graph together with explicit desiderata — importance retention, atomicity, causal-information preservation, and interpretability — and an objective that makes them measurable.
- It proposes an automated three-stage pipeline — pruning, clustering, and visualizing — that produces a concise summary graph that is faithful to the original attribution graph.
- It evaluates the method on the BATS analogy and multi-hop reasoning datasets, showing through steering interventions that the resulting summaries are causally faithful and that the supernodes behave as the single mechanisms.

1.5 Organization of Thesis

The remainder of this thesis is organized as follows. Chapter 2 reviews the background the method depends on, tracing the line from superposition and sparse dictionary learning to the cross-layer transcoders and circuit-tracing pipeline that

produce attribution graphs, and surveys the related work on describing and summarizing circuits from which the thesis takes its motivation. Chapter 3 presents the proposed method, formalizing the summary-graph problem and detailing its three stages of pruning, clustering, and visualizing. Chapter 4 evaluates the method empirically, defining the faithfulness metrics, reporting the pruning and clustering comparisons, and presenting the steering-intervention study that validates the summary graphs against the underlying model. Chapter 5 discusses the findings and their limitations and concludes with directions for future work.

CHAPTER 2. LITERATURE REVIEW

The previous chapter established that an attribution graph gives a faithful, mechanistic account of how a language model produces a single prediction, but that inspecting such graphs by hand does not scale, which motivates an automated method for summarizing them. Designing that method requires a precise grasp of the objects it manipulates and the techniques it builds upon, and this chapter supplies that background. It first bounds the scope of the work (Section 2.1), then reviews the transformer architecture (Section 2.2) and the concepts of mechanistic interpretability (Section 2.3), the sparse autoencoders and transcoders that recover interpretable features (Section 2.4), and the circuit-tracing backbone of Ameisen et al. [1] that produces the attribution graphs this thesis summarizes (Section 2.5). It closes with the automated interpretation methods that supply feature labels (Section 2.6) and a review of related work on describing and summarizing circuits (Section 2.7).

2.1 Scope of Research

This thesis studies the post-hoc mechanistic interpretation of a single decoder-only language model through the attribution graph of one prompt at a time. It takes the replacement model and the raw attribution graph of Ameisen et al. [1] as given, and concentrates on summarizing that graph into a compact, faithful, and human-readable form. Training the underlying transcoders, recovering circuits that run through attention patterns, and reasoning jointly across many prompts at once all lie outside its scope. Feature meanings are taken from existing Neuronpedia descriptions rather than generated anew.

2.2 Transformer

The transformer [4] maps a sequence of token embeddings to next-token predictions through a stack of attention and MLP sublayers. The decoder-only models studied here share one structural property that makes them amenable to mechanistic analysis: every sublayer reads from a shared *residual stream* and adds its output back into it instead of overwriting it (Figure 2.1). The representation at any depth is therefore the sum of the original embedding and the contributions of all preceding sublayers, and the output logits are a linear read-out of this sum. This additive structure is what allows a later computation to be attributed back to the specific earlier contributions that produced it, and it is the structure the replacement model of Section 2.5 exploits. The internal mechanics of attention and the MLP are not needed in what follows; we refer the reader to the original work for them.

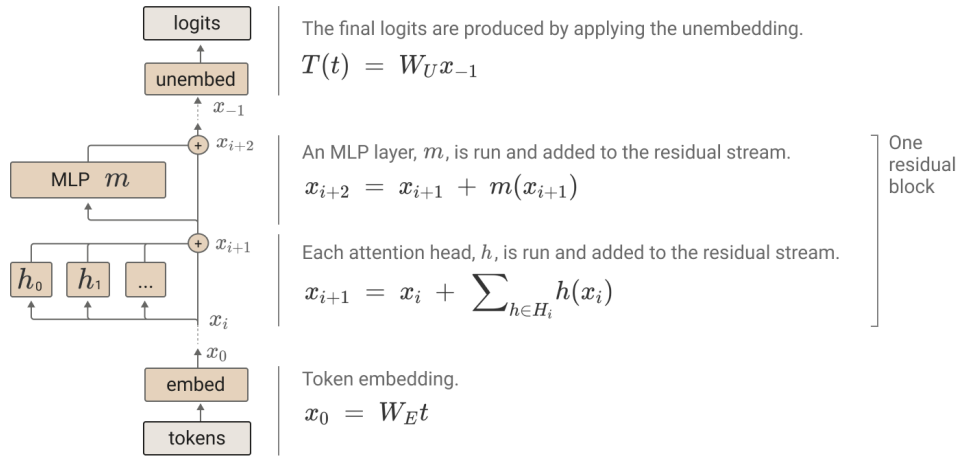


Figure 2.1: The decoder-only transformer as a residual network. Each sublayer reads the residual stream and adds its output back into it, so the final representation is a sum of contributions from the embedding, every attention head, and every MLP.

2.3 Mechanistic Interpretability and Circuits

Mechanistic interpretability is a subfield of explainable AI that seeks to reverse-engineer a model’s internal computation into human-interpretable mechanisms, recovering the algorithm the model implements rather than only relating its inputs to its outputs. Its basic unit is the *feature*: a property of the input that the model computes and reuses across contexts, such as a concept or a syntactic role. The *linear representation hypothesis* holds that a feature is a direction in activation space whose strength is the magnitude of the activation along it [5]; this is natural for transformers, whose computation is dominated by linear maps on the residual stream, so meaningful properties can be encoded as directions and recovered by projection.

A *circuit* is a subgraph of the model’s computation that maps earlier features to later ones through the specific weights and non-linearities used in that mapping [6]. Features compose hierarchically, with early layers computing simple features that later layers combine into more abstract ones, and the residual stream lets a circuit span non-adjacent layers by carrying features forward unchanged. Recovering circuits expressed over interpretable features is the central goal of this thesis, because such a circuit explains not merely *what* a model predicts but *how* the prediction is computed.

The obstacle to reading circuits directly off neurons is *superposition*: a model represents more features than it has dimensions, packing them as an overcomplete set of nearly orthogonal directions in activation space [7]. A direct consequence is *polysemanticity*, where a single neuron activates for several unrelated features and so admits no single interpretation. Superposition is an efficient compression

strategy for the model but a barrier for interpretation, because the computational basis of neurons no longer aligns with the representational basis of features. Overcoming this misalignment is what the dictionary-learning methods of the next section are designed to do.

2.4 Sparse Autoencoders and Transcoders

Sparse autoencoders (SAEs) address the polysemanticity barrier by learning a representational basis that is aligned with features rather than neurons [8]. Framed as dictionary learning, an SAE encodes a model activation $x \in \mathbb{R}^d$ into a sparse, overcomplete code $c = \text{ReLU}(W_{\text{enc}}x + b)$ with far more entries than d , and reconstructs the activation from that code as $\hat{x} = W_{\text{dec}}c$. Training minimises the reconstruction error $\|x - \hat{x}\|_2^2$ together with an ℓ_1 sparsity penalty $\alpha\|c\|_1$, in which α trades reconstruction fidelity against sparsity. Because only a few code units are active at a time, each unit can specialise to a single feature, and the resulting dictionary directions are substantially more monosemantic and interpretable than individual neurons. SAEs thus recover, in an unsupervised way, the feature directions that superposition had hidden.

This feature basis is not always one-to-one with human concepts. Chanin et al. [9] show that hierarchical concepts in SAEs can undergo feature splitting, where a coarse feature separates into finer child features, and feature absorption, where the expected parent feature fails to activate because its signal is absorbed by those children. Thus, one semantic concept may be distributed across multiple learned features, which matters when grouping feature nodes into higher-level circuit units.

SAEs nevertheless remain awkward for circuit analysis, because they are trained to reconstruct one activation in place and say nothing directly about how features at one site cause features at another. A *transcoder* is a variant designed to close this gap [10]: instead of reconstructing its input, it predicts the *output* of an MLP sublayer from that sublayer’s *input*. With the same sparse encoder $z(x) = \text{ReLU}(W_{\text{enc}}x + b_{\text{enc}})$ and a decoder $\text{TC}(x) = W_{\text{dec}}z(x) + b_{\text{dec}}$, a transcoder is trained to approximate the MLP map,

$$\mathcal{L}_{\text{TC}}(x) = \|\text{MLP}(x) - \text{TC}(x)\|_2^2 + \lambda\|z(x)\|_1. \quad (2.1)$$

By replacing the MLP with a sparse linear map over interpretable features, the transcoder *skips over* the MLP non-linearity that otherwise blocks attribution. The effect of one feature on another then factorises into an input-dependent term, the source activation, and an input-invariant term given by the dot product between the source decoder direction and the target encoder direction, which depends only on the learned weights. This factorisation is what makes feature-to-feature edges

cheap and faithful to compute, and it is the property the circuit-tracing pipeline of the next section is built upon.

2.5 Circuit Tracing Backbone

This thesis takes as its input the attribution graphs produced by the circuit-tracing methodology of Ameisen et al. [1], so this section describes that methodology at the level needed to understand what the summarization pipeline consumes. A companion case-study paper applies the same attribution-graph workflow to Claude 3.5 Haiku, using pruned and grouped graphs to inspect prompt-specific mechanisms [11]. This section proceeds in three parts: the replacement model that re-expresses the transformer as a linear network over interpretable features; the attribution graph built from that model for a single prompt; and the interventions that test whether the graph is faithful. The pruning and supernode grouping steps that this thesis automates appear here only as the manual baseline they replace, their formal treatment being deferred to the methodology chapter.

2.5.1 Replacement Model

The replacement model re-expresses the underlying transformer as a network whose computation runs over interpretable features rather than neurons, so that the effect of one feature on another can be read off directly. It is built from cross-layer transcoders and is then specialized to a single prompt.

a, Cross-Layer Transcoders

A cross-layer transcoder (CLT) extends the single-layer transcoder of Section 2.4 so that a feature can act on several layers at once [1]. As before, features read from the residual stream through a sparse encoder, but a feature that activates at layer ℓ' now writes to the output of its own MLP *and* of every later MLP. Writing x^ℓ for the residual stream entering the MLP of layer ℓ , the feature activations and the reconstructed MLP output are

$$a^\ell = \text{JumpReLU}(W_{\text{enc}}^\ell x^\ell), \quad (2.2)$$

$$\hat{y}^\ell = \sum_{\ell' \leq \ell} W_{\text{dec}}^{\ell' \rightarrow \ell} a^{\ell'}. \quad (2.3)$$

A single CLT is trained to reconstruct every MLP output across the model under a sparsity penalty, leaving on the order of a hundred active features per token. Because a feature can deposit its effect many layers downstream without an intervening reconstruction step, the CLT recovers genuinely cross-layer features and avoids the near-duplicate features that a stack of independent single-layer transcoders would produce at each layer.

b, From Global to Local Replacement Model

The global replacement model substitutes the CLT for every MLP in the network while leaving the attention layers untouched, giving a single model that computes over features at every layer. This substitution is imperfect: the CLT does not reconstruct the MLP outputs exactly, and the resulting model reproduces the original next-token prediction on only about half of pretraining-style prompts. The global model is therefore not analyzed directly; it serves as the starting point for a prompt-specific construction in which the two remaining sources of nonlinearity are removed.

For a fixed prompt, the local replacement model freezes the two input-dependent nonlinear components — the attention patterns and the LayerNorm normalization denominators — to the values they took on the original forward pass, and adds an *error node* at each (layer, token) position equal to the difference between the true MLP output and the CLT reconstruction there. The error terms make the construction exact: with them, the local replacement model reproduces the underlying model’s activations and logits on that prompt. The freezing makes it linear: once attention and normalization are constants, each feature’s preactivation is an affine function of the upstream feature activations, so a feature’s input is simply the sum of its incoming edges. This linearity is the property the rest of the methodology rests on, because it lets the effect of one feature on another be attributed exactly rather than estimated.

2.5.2 Attribution Graphs

An attribution graph is the directed graph of feature-to-feature effects that the local replacement model induces for one prompt. Its nodes are the active features and the boundary terms of that model, and its edges are the direct linear effects between them. Figure 2.2 shows the dashboard view used to inspect such graphs and their selected subgraphs.

a, Nodes

The graph has four kinds of nodes. *Feature nodes* are the active CLT features, each tied to a (layer, token) position and carrying an interpretable meaning; they are the nodes that the summarization pipeline groups. *Embedding nodes* hold the prompt’s token embeddings and act as primary inputs with no incoming edges. *Error nodes* hold the per-position reconstruction residuals; like embedding nodes they are sources with no incoming edges, but they carry no interpretation and stand for the part of the computation the CLT failed to capture. *Logit nodes* represent the candidate output tokens and act as sinks with no outgoing edges. The methodology chapter formalizes these four node types as the vertex set on which the summary-

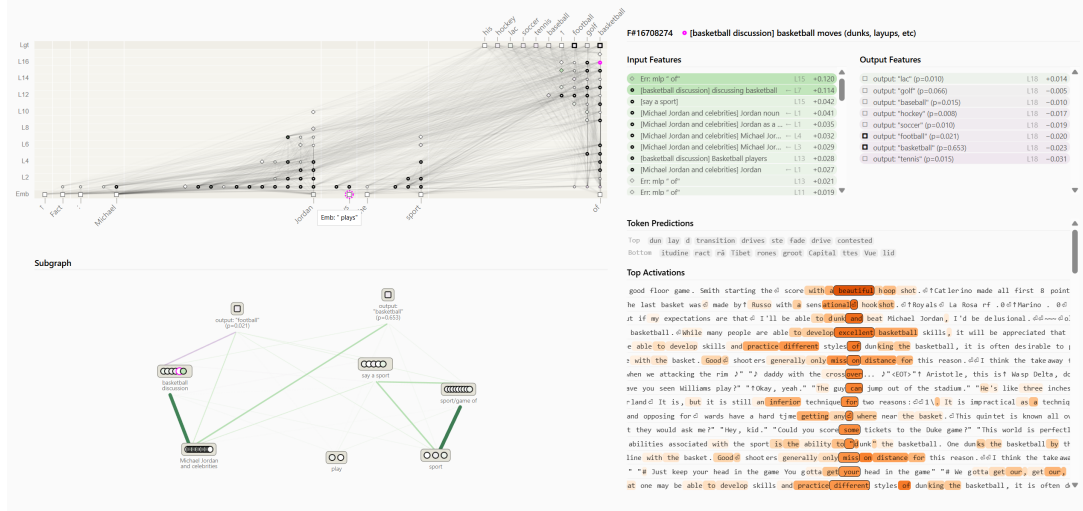


Figure 2.2: Dashboard visualization of an attribution graph. The interface shows the graph over prompt positions and layers, lists input and output features, and displays a selected subgraph used for circuit inspection.

graph problem is defined.

b, Edges and Edge Weights

An edge measures the direct linear effect of a source node on a target feature’s preactivation through the frozen local replacement model, where “direct” means the immediate interaction, with any effect mediated by other features counted as separate edges. Because the model is linear on a fixed prompt, this effect factorizes into the prompt-dependent source activation and an input-invariant virtual weight,

$$A_{s \rightarrow t} = a_s w_{s \rightarrow t}. \quad (2.4)$$

The virtual weight $w_{s \rightarrow t}$ is the dot product of the source feature’s decoder vector with the target feature’s encoder vector, summed over the layers at which the source writes ahead of the target; for an embedding source the embedding vector replaces the decoder, and for a logit target the unembedding replaces the encoder. Since a_s depends on the prompt while $w_{s \rightarrow t}$ depends only on the trained weights, each edge is cheap to evaluate and faithful to the model’s computation. Collecting these signed weights over all node pairs gives the weighted adjacency matrix that the next chapter takes as its input.

c, Pruning and Supernode Grouping in the Original Method

The raw attribution graph is too large to read: it holds thousands of feature nodes joined by a dense web of edges, most of which carry negligible weight. Ameisen et al. reduce it in two manual steps. They first prune the graph, keeping only the nodes and edges that account for most of the indirect influence on the logits

and discarding the rest. They then group the surviving features into supernodes by inspection, collecting features that play the same role into a single labelled unit so the circuit can be drawn as a small diagram. Both steps rely on human judgment and do not scale: pruning thresholds are tuned by hand, and grouping a single circuit can take hours of expert effort. Automating exactly these two steps is the contribution of this thesis; the methodology chapter formalizes pruning as the pruning stage (Section 3.3.1) and supernode grouping as the clustering and visualizing stages (Sections 3.3.2–3.3.3).

Even after this reduction, the graph remains difficult to interpret. Pruned graphs can still contain hundreds of features and thousands of signed edges on short prompts, so size is only one part of the obstacle. The mechanism of interest may be distributed across several features, each feature may collect many weak inputs from diverse upstream sources, and the same downstream influence may be carried through several paths with different lengths or even opposite signs. These properties make a node-by-node explanation hard to summarize manually, and they motivate an automated summary that groups role-equivalent features while preserving signed causal structure.

2.5.3 Interventions

An attribution graph is a hypothesis about how the model computes its output, and the same local replacement model supplies the means to test it. The central tool is *constrained activation patching*: a feature’s activation is set to a chosen value over the narrow layer window in which it acts, the forward pass is resumed from that point, and the resulting changes in other features and in the output logits are measured. Steering a feature — for example multiplying its activation by a negative factor to suppress it — and checking that the downstream effects match those the graph predicts confirms that the recovered edges describe genuine causal pathways rather than coincidental correlations. These interventions are how the faithfulness of a circuit is established, and this thesis reuses them in its evaluation to verify that the supernodes of a summary graph behave as the single mechanisms their labels claim.

2.6 Automated Interpretation Methods

Recovering a feature direction leaves open what it means, and labelling features by hand does not scale to the millions a dictionary can contain. Automated interpretation (auto-interp) assigns each unit a natural-language description with a large language model: an *explainer* model writes a description from the unit’s top-activating examples, and a *simulator* model is then scored on how well that description predicts the

unit’s activations on held-out text [12]. Because the score rewards descriptions that capture the true activation pattern, auto-interp succeeds far better on the monosemantic features of sparse autoencoders and transcoders than on polysemantic neurons, whose many unrelated activations no single sentence can describe. This thesis does not run auto-interp itself; it consumes the resulting public feature labels through Neuronpedia to name the supernodes of a summary graph.

2.7 Related Works

The work closest to this thesis automates the interpretation of attribution graphs directly. ADAG [3] introduces attribution profiles to quantify the functional role of neurons and clusters nodes using those profiles. However, it does not explicitly optimize the structural causal-preservation objective used here.

A parallel line of work questions the substrate on which circuits are built. Arora et al. [2] show that circuits traced directly in the neuron basis can be as sparse and as faithful as those built from transcoder features, avoiding transcoder training, reconstruction error, and the associated error nodes. The approach is attractive for those reasons, but the authors observe that the resulting circuits remain too large for a human to read and call for principled grouping methods to compress them. That is precisely the bottleneck this thesis targets, and the summarization method developed here is agnostic to the substrate: it operates on any signed attribution graph, whether its nodes are transcoder features or neurons.

Summarizing large directed graphs is also studied outside interpretability. Causal DAG summarization [13] compresses the causal graphs used in causal inference while preserving their salient relationships, offering principled objectives for graph summarization. Those objectives do not transfer directly, because a causal-inference DAG encodes relationships among external variables rather than signed, feature-to-feature effects within a model on a specific prompt, and it carries no notion of faithfulness to an underlying network. Across these lines of work, no existing method produces a compact and faithful structural summary of an attribution graph by jointly pruning and clustering its feature nodes under an explicit faithfulness objective; supplying one is the aim of this thesis.

This chapter assembled the background the summarization pipeline depends on, following a single line of reasoning. Superposition hides a model’s features inside its neurons; sparse autoencoders and transcoders recover those features as interpretable directions; and the cross-layer transcoders of the circuit-tracing backbone turn them into a per-prompt attribution graph whose edges are exact linear effects between features. That attribution graph is the object this thesis summarizes. The

review also located the gap the thesis fills: the original methodology prunes and groups the graph by hand, while related work either describes graphs without compressing them or produces circuits that are still too large to read. Automated structural summarization of an attribution graph therefore remains open, and the next chapter formalizes that problem and presents the three-stage method — pruning, clustering, and visualizing — that solves it.

CHAPTER 3. METHODOLOGY

3.1 Overview

Chapter 2 established the foundation of the circuit tracing backbone that construct the attribution graphs, and the difficulty of reading them. Chapter 3 presents the proposed automated summarization method for turning such an attribution graph into a smaller *summary graph* whose nodes represent groups of related features and whose edges preserve the main causal flow. The method has three stages. **Pruning** removes unimportant attribution-graph nodes and edges. **Clustering** groups functionally similar features into supernodes, further reduce the complexity while preserving causal information. **Visualizing** aggregates the clustered graph into a directed acyclic summary graph and assigns labels to its supernodes. Section 3.2 formalizes the input, output, and the desiderata of the summary. Section 3.3 then defines the three stages of the method in sequence. Figure 3.1 gives an overview of the pipeline on the example where the model was given the prompt “Fact: The capital of the state containing Dallas is”.

3.2 Problem Formulation

3.2.1 Input

The attribution graph is a Directed Acyclic Graph (DAG) $G = (V, E, W)$ for one prompt and one model prediction. We name the non-error node sets as

$$V = V_{\text{emb}} \cup V_{\text{feature}} \cup V_{\text{logit}}$$

Embedding nodes represent input-token sources, feature nodes represent transcoder features and logit nodes represent output targets. The weighted adjacency matrix $W \in \mathbb{R}^{N \times N}$, with $N := |V|$, follows the convention that W_{ij} is the signed linear influence from source node i to target node j . Embedding nodes are sources with no incoming edges, logit nodes are sinks with no outgoing edges, and feature nodes carry signal between them.

The input also include extra information of feature nodes such as: max activating contexts, top logits, activation frequency.

The token weight vector $\tau \in \mathbb{R}^N$ marks the input tokens of interest and satisfies $\sum_e \tau_e = 1$ over the selected embedding nodes. The logit weight vector $\ell \in \mathbb{R}^N$ marks the output positions or target logits of interest. The vector τ is initialized using normalized SHAP values. For a multi-target summary, ℓ is initialized using logit probabilities; for a single-target summary, it assigns weight 1 to the target

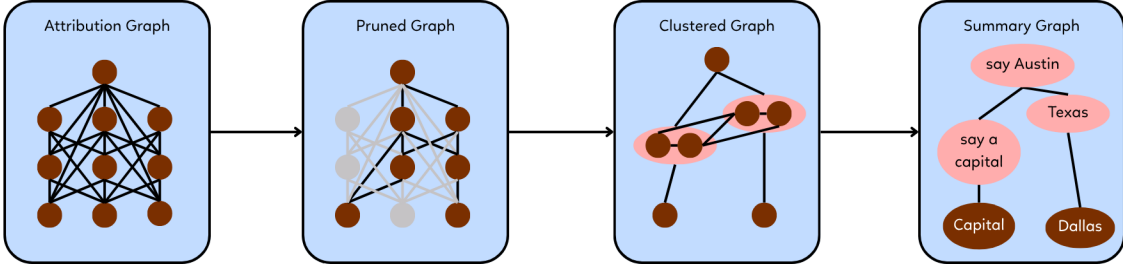


Figure 3.1: Methodology pipeline. The method starts from a dense attribution graph, prunes it to the main causal pathways, clusters related feature nodes into supernodes, then removes cycle-causing links and assigns readable labels so the final summary can be inspected as a causal story.

logit and weight 0 to all other nodes. This thesis mainly studies the single-target case.

3.2.2 Output

The output is a *summary graph* $H = (V_H, E_H, W_H)$. Its node set $V_H = \{C_1, C_2, \dots, C_K\}$ consists of *supernodes*, where each supernode is a nonempty subset of the attribution-graph nodes:

$$V_H \subseteq \mathcal{P}(V) \setminus \{\emptyset\}. \quad (3.1)$$

Writing $U_H := \bigcup_{C \in V_H} C$ for the attribution-graph nodes covered by the summary, admissible supernodes satisfy

$$C_i \cap C_j = \emptyset \quad (i \neq j), \quad \forall v \in U_H \cap (V_{\text{emb}} \cup V_{\text{logit}}), \{v\} \in V_H. \quad (3.2)$$

Therefore V_H forms a partition of the represented subset of non-error nodes $U_H \subseteq V$. Embedding and logit nodes are represented only as singleton supernodes, so only feature nodes may be grouped. The edge set E_H and signed weights W_H describe the net causal influence between supernodes.

3.2.3 Desiderata of the summary graph

A useful summary graph scores high in both interpretability and faithfulness, that is it must be small enough to inspect while still representing the causal computation of the original attribution graph. Four desiderata make this requirement explicit.

(D1) Importance retention.

The summary should retain the nodes and edges that lie on high-mass pathways from the selected input tokens to the selected logits, and discard nodes and edges that do not contribute to those pathways.

(D2) Atomicity.

Each supernode should represent one causal role. Its member features should

have similar signed downstream targets and similar signed upstream sources, so the reader can interpret the supernode as a single mechanism.

(D3) Causal-information preservation.

Causal relations between retained nodes should remain visible as edges between supernodes when possible. If two directly connected nodes are merged into one supernode, their edge becomes internal and no longer appears in H .

(D4) Interpretability.

The final graph should be a directed acyclic graph with few enough supernodes for manual inspection. Acyclicity is required because the summary is intended to show a forward causal story.

3.3 Method

3.3.1 Pruning

Since the attribution graph contains redundant nodes and edges, the author of [1] proposed a pruning algorithm to keep nodes and edges that directly or indirectly exert significant influence on the logit nodes. However, it failed to bring the number of nodes and edges down to interpretable range, because even at low thresholds, the pruned graph still contains noisy features, activated from punctuation, stopwords, format scaffolding, ... We usually care more about how the model derives the semantics required to output the next token rather than the syntactic features. Our pruning stage targets D1: it keeps nodes and edges that participate in the selected input-to-output computation, and removes the rest so that later stages operate on a tractable subgraph. The pruning rule is based on two path-reachability scores, influence and relevance, followed by a retained-mass selection rule.

Influence and relevance. Influence measures how strongly a node can reach the selected logits, while relevance measures how strongly a node is reached from the selected input tokens. Let

$$W_{ij}^{\text{in}} = \begin{cases} \frac{|W_{ij}|}{\sum_k |W_{kj}|}, & \text{if } \sum_k |W_{kj}| > 0, \\ 0, & \text{otherwise,} \end{cases} \quad W_{ij}^{\text{out}} = \begin{cases} \frac{|W_{ij}|}{\sum_k |W_{ik}|}, & \text{if } \sum_k |W_{ik}| > 0, \\ 0, & \text{otherwise} \end{cases}$$

be the column-normalized and row-normalized absolute adjacency matrices. The per-node influence and relevance vectors are

$$\text{Inf} = \ell^\top (I - (W^{\text{in}})^\top)^{-1} = \ell^\top \sum_{k \geq 0} ((W^{\text{in}})^\top)^k \in \mathbb{R}^N, \quad (3.3)$$

$$\text{Rel} = \tau^\top (I - W^{\text{out}})^{-1} = \tau^\top \sum_{k \geq 0} (W^{\text{out}})^k \in \mathbb{R}^N. \quad (3.4)$$

Because G is a DAG, both normalized adjacency matrices are nilpotent after a topological ordering of the nodes. The Neumann series therefore terminates after finitely many terms, so the inverse expressions are well-defined.

Edge influence and relevance are computed after node pruning, so that edge scores are assigned only inside the node-pruned graph. Let $\tilde{W}$ be the matrix obtained by setting the rows and columns of unretained nodes to zero, and let $\tilde{W}^{\text{in}}$ and $\tilde{W}^{\text{out}}$ be its column-normalized and row-normalized absolute adjacency matrices. The node influence and relevance vectors used for edge scoring are recomputed on this node-pruned graph:

$$\begin{aligned} \text{Inf}^{\tilde{G}} &= \ell^\top \sum_{k \geq 0} ((\tilde{W}^{\text{in}})^\top)^k, \\ \text{Rel}^{\tilde{G}} &= \tau^\top \sum_{k \geq 0} (\tilde{W}^{\text{out}})^k. \end{aligned}$$

For an edge (i, j) from source node i to target node j , the edge influence and edge relevance are then

$$\text{Inf}_{ij}^{\text{edge}} = \tilde{W}_{ij}^{\text{in}} \text{Inf}_j^{\tilde{G}}, \quad (3.5)$$

$$\text{Rel}_{ij}^{\text{edge}} = \tilde{W}_{ij}^{\text{out}} \text{Rel}_i^{\tilde{G}}. \quad (3.6)$$

Thus, edge influence distributes the downstream influence of a target node across its retained incoming edges, while edge relevance distributes the upstream relevance of a source node across its retained outgoing edges.

Importance score. In [1], the author only used influence to quantify nodes and edges importance. We redefine this assumption saying a node is important only when both scores are high: it must be connected to the selected input context and also influence the selected output. After normalizing influence and relevance to

$[0, 1]$ across nodes, the node importance score is

$$\text{Imp}_v = \widehat{\text{Inf}}_v^\alpha \widehat{\text{Rel}}_v^{1-\alpha}, \quad \alpha \in [0, 1], \quad (3.7)$$

where $\widehat{\cdot}$ denotes the rank normalized score. The geometric mean is used because it assigns a low score to nodes with small influence or relevance. Edges are scored by applying the same influence–relevance principle to their edge scores:

$$\text{Imp}_{ij} = (\widehat{\text{Inf}}_{ij}^{\text{edge}})^\alpha (\widehat{\text{Rel}}_{ij}^{\text{edge}})^{1-\alpha}, \quad \alpha \in [0, 1], \quad (3.8)$$

where the normalization is applied across all candidate edges.

Threshold selection The pruning stage keeps the highest-importance nodes whose cumulative importance mass reaches a retained-mass fraction $\eta_V \in [0, 1]$, and keeps edges by the analogous retained-mass fraction η_E . Equivalently, if V' and E' are the retained nodes and edges, they are chosen so that

$$\frac{\sum_{v \in V'} \text{Imp}_v}{\sum_{v \in V} \text{Imp}_v} \geq \eta_V, \quad (3.9)$$

$$\frac{\sum_{(u,v) \in E'} \text{Imp}_{uv}}{\sum_{(u,v) \in E} \text{Imp}_{uv}} \geq \eta_E. \quad (3.10)$$

The implementation obtains these sets by sorting nodes and edges by importance in descending order and keeping the smallest prefix whose cumulative importance mass reaches the corresponding fraction. Larger values of η_V and η_E retain more of the original graph before clustering. We use default thresholds η_V and η_E at 0.02 and 0.95 respectively, the choice will be explain in chapter Numerical Results.

Filter dead features and too frequent features Because the transcoders are not perfect, they often include features that activate too frequent or too infrequent. A feature’s activation frequency is the fraction of tokens in a dataset for which the feature has positive activation. Dead features have activation frequency near 0, while some features activate too frequently. We discard features with activation frequency below 0.002% or above 10%. The choice of thresholds is arbitrary.

3.3.2 Clustering

The clustering stage targets D2 and D3 in the desiderata. We formulate it as a binary integer linear program (ILP) over the retained feature nodes. The program decides which pairs of features belong to the same supernode, while embedding and logit nodes are kept outside the optimization as singleton supernodes. This

formulation makes the design explicit: the objective rewards grouping features with similar signed causal roles, and the constraints enforce a valid partition, a causal-information budget, a complexity budget, and a layer-span restriction.

Feature representation. Let $F = V'_{\text{feature}} = \{1, \dots, n\}$ be the retained feature nodes after pruning, and let W' denote the pruned adjacency matrix induced by retained nodes V' and retained edges E' . A feature's causal role is represented by its signed outgoing and incoming edge profiles. For a retained feature node $u \in F$, define

$$\begin{aligned} (v_u^{\text{out}})_k &= W'_{uk} \sqrt{\text{Inf}_k}, & k \in V', \\ (v_u^{\text{in}})_k &= W'_{ku} \sqrt{\text{Rel}_k}, & k \in V', \\ r(u) &= [v_u^{\text{out}}; v_u^{\text{in}}] \in \mathbb{R}^{2|V'|}. \end{aligned} \quad (3.11)$$

The outgoing half records which downstream nodes u influences, weighted by the influence of those targets. The incoming half records which upstream nodes influence u , weighted by the relevance of those sources. The coordinates keep the sign of W' , so features that affect the same targets with opposite signs have opposing role vectors rather than appearing similar.

ILP data. The ILP is built from pairwise constants computed before optimization. The signed role similarity between two retained features is

$$s_{ij} = \cos(r(i), r(j)) \in [-1, 1], \quad i, j \in F. \quad (3.12)$$

A resolution threshold θ converts this similarity into a linear merge cost $c_{ij}^{\text{atom}} = \theta - s_{ij}$. If $s_{ij} > \theta$, then $c_{ij}^{\text{atom}} < 0$ and the ILP is rewarded for merging the pair; if $s_{ij} < \theta$, then $c_{ij}^{\text{atom}} > 0$ and the ILP is penalized for merging the pair. The implementation uses an adaptive percentile threshold θ , which is the q -th percentile of the allowed-pair cosine values on the current graph. A small threshold will incentivize more merging, while a large threshold will incentivize less merging. We found that a threshold of 65% works well for most graphs, higher threshold might create many singleton supernodes which increase the complexity of the summary graph.

The causal-preservation coefficient of a pair is the retained edge mass that would be hidden if the two features were placed in the same supernode:

$$c_{ij}^{\text{causal}} = \frac{|W_{ij}| + |W_{ji}|}{\sum_{(u,v) \in E'} |W_{uv}|}, \quad i, j \in F, \quad i < j. \quad (3.13)$$

Here both directions are included because either edge becomes internal after a merge. If the denominator is zero, the causal budget is inactive. A layer-span constant $\Delta_{\max}$ defines the allowed-pair set

$$\mathcal{P} = \{(i, j) : i, j \in F, i < j, |\text{layer}(i) - \text{layer}(j)| \leq \Delta_{\max}\}. \quad (3.14)$$

Pairs outside $\mathcal{P}$ are not allowed to share a supernode.

Decision variables. The ILP uses binary pair variables

$$x_{ij} \in \{0, 1\}, \quad (i, j) \in \mathcal{P}, \quad (3.15)$$

where $x_{ij} = 1$ means that feature nodes i and j are assigned to the same supernode. To count supernodes, the model also uses binary representative variables

$$r_i \in \{0, 1\}, \quad i \in F, \quad (3.16)$$

where $r_i = 1$ means that i is the lowest-indexed feature in its supernode. The number of feature supernodes is therefore $K_F = \sum_{i \in F} r_i$.

ILP objective. The objective minimizes the signed-cosine atomicity loss:

$$\min_{x, r} \sum_{(i, j) \in \mathcal{P}} (\theta - s_{ij}) x_{ij}. \quad (3.17)$$

This is a linear objective because all similarities and the threshold are fixed before solving. The formulation does not place L_{causal} in the objective with a trade-off weight. Instead, causal preservation is enforced as a hard constraint, so the solver first respects the permitted hidden edge mass and then chooses the most atomic partition inside that feasible region.

Partition constraints. The pair variables must describe an equivalence relation. For any three features i , j , and k , transitivity is enforced by triangle inequalities such as

$$x_{ij} + x_{jk} - x_{ik} \leq 1, \quad x_{ij} + x_{ik} - x_{jk} \leq 1, \quad x_{ik} + x_{jk} - x_{ij} \leq 1, \quad (3.18)$$

whenever the corresponding pair variables are present. If one pair is outside $\mathcal{P}$, the same rule becomes a layer-span exclusion; for example, if $(i, k) \notin \mathcal{P}$, then $x_{ij} + x_{jk} \leq 1$. These constraints prevent the ILP from assigning i with j and j with

k while keeping i and k in different supernodes.

Representative constraints. The representative variables count one and only one lowest-indexed member for each supernode. For each feature i , let $A_i = \{j < i : (j, i) \in \mathcal{P}\}$ be the allowed earlier features. The constraints are

$$r_i + x_{ji} \leq 1, \quad i \in F, j \in A_i, \quad (3.19)$$

$$r_i + \sum_{j \in A_i} x_{ji} \geq 1, \quad i \in F. \quad (3.20)$$

If feature i is merged with an earlier feature, Eq. (3.19) forces $r_i = 0$. If it is not merged with any earlier feature, Eq. (3.20) forces $r_i = 1$. Hence $K_F = \sum_i r_i$ counts feature supernodes without requiring explicit cluster labels.

Budget constraints. The causal-information budget limits the retained edge mass hidden inside supernodes:

$$\sum_{(i,j) \in \mathcal{P}} c_{ij}^{\text{causal}} x_{ij} \leq \varepsilon_{\text{causal}}, \quad 0 \leq \varepsilon_{\text{causal}} \leq 1. \quad (3.21)$$

The complexity budget limits the number of feature supernodes:

$$\sum_{i \in F} r_i \leq K_{\max}. \quad (3.22)$$

The first budget preserves visible causal edges by preventing too much retained edge mass from becoming internal to supernodes. The second budget keeps the summary readable by bounding the number of feature supernodes before embedding and logit singleton supernodes are added back.

Cluster recovery. After the ILP is solved, the feature partition is recovered from the selected pair variables: all pairs with $x_{ij} = 1$ are joined, and the connected components under these joins become feature supernodes. The transitivity constraints ensure that these components define a valid partition. Embedding and logit nodes are then added as singleton supernodes, so the clustering output satisfies the output conditions of Eq. (3.2).

3.3.3 Visualizing

Visualizing converts the clustered pruned graph into the final summary graph. This stage takes the partition f from clustering, aggregates retained edges between supernodes, removes cycles introduced by aggregation, and assigns human-readable

labels. It ensures D4 is satisfied, so that a human reader can understand the summary clearly.

Block-sum adjacency. For each ordered pair of distinct supernodes (C, C') , the signed block-sum edge weight is

$$B_{C \rightarrow C'} = \sum_{u \in C, v \in C'} W_{uv} \mathbf{1}[(u, v) \in E']. \quad (3.23)$$

This quantity sums all retained attribution-graph edges from members of C to members of C' , where W_{uv} is the edge from source u to target v . Diagonal blocks are omitted because edges inside a supernode are represented as part of that node rather than as visible edges in H .

Edge direction The first acyclicity step removes two-node cycles. For each unordered pair $\{C, C'\}$, let $a = B_{C \rightarrow C'}$ and $b = B_{C' \rightarrow C}$. The method keeps only the net flow in the direction with larger magnitude:

$$(B_{C \rightarrow C'}, B_{C' \rightarrow C}) \leftarrow \begin{cases} ((|a| - |b|) \operatorname{sgn}(a), 0) & \text{if } |a| > |b|, \\ (0, (|b| - |a|) \operatorname{sgn}(b)) & \text{if } |b| > |a|, \\ (0, 0) & \text{if } |a| = |b|. \end{cases} \quad (3.24)$$

This rule removes every two-node cycles and leaves an edge only when one direction has nonzero net dominance.

Back-edge removal. The second acyclicity step removes longer cycles by imposing a depth order on the supernodes. Embedding supernodes receive depth $-\infty$, logit supernodes receive depth $+\infty$, and each feature supernode receives the median layer of its member features. Ties are broken by mean member layer, then by the smallest member `ctx_idx`, and finally by supernode index. After sorting by this order, the method deletes every edge $C \rightarrow C'$ whose source ranks after its target. The remaining adjacency is the summary adjacency W_H , and the final edge set is

$$E_H = \{(C, C') : W_{H, C \rightarrow C'} \neq 0\}.$$

By construction, all surviving edges point forward in the imposed order, so H is a DAG.

Labeling. The labeling step uses an LLM to translate the supernodes into human-interpretable concepts. We label the supernodes by adapting previous automatic-interpretability techniques [12] to better fit our pipeline. The implementation provides the LLM with the original prompt, the predicted token, and compact feature evidence for each supernode, including its layer, token position, activation examples, neighboring tokens, and top positive logits. For each supernode, the output includes a short label, a role, and a one-sentence description. We divide the supernodes into four roles:

Input.

Low-level features that mainly extract evidence from the prompt.

Output.

Features that promote the prediction of specific tokens.

Abstract.

High-level concepts and intermediate computations.

Trash.

Uninterpretable features.

This taxonomy follows [11] and fits the detokenization–abstract-features–retokenization picture of Elhage et al. [14].

3.4 Chapter Summary

This chapter formalized attribution-graph summarization and defined the proposed three-stage method. The input is a signed directed acyclic attribution graph, and the output is a directed acyclic summary graph whose supernodes partition a retained subset of nodes. The method first prunes the attribution graph using influence, relevance, and the pruning loss L_{prune} . It then clusters retained feature nodes by a binary ILP whose pair variables encode same-supernode assignments. The ILP minimizes a signed-cosine atomicity objective subject to transitivity, representative-counting, causal-information, complexity, and layer-span constraints. Finally, it aggregates the clustered graph with block-sum edge weights, removes cycles, and labels the resulting supernodes for inspection. The next chapter analyzes properties of this construction, and Chapter 5 evaluates whether the resulting summary graphs are faithful to the replacement model under steering interventions.

CHAPTER 4. NUMERICAL RESULTS

4.1 Overview

The previous chapter presented the proposed summarization pipeline. This chapter evaluates whether the proposed summarization pipeline preserves the causal structure of attribution graphs while making them smaller and easier to interpret. The evaluation is organized around three questions. First, does influence–relevance pruning retain enough of the replacement-model computation to remain faithful to the original attribution graph? Second, does the proposed ILP clusterer produce supernodes whose members have coherent downstream causal effects? Third, do the final summary nodes and edges predict the behavior of the model under intervention?

The chapter is therefore organized by experiment rather than by separating all metrics, baselines, and results into different sections. Section 4.2 describes the common datasets, implementation environment, and default pipeline configuration. Section 4.3 evaluates pruning. Section 4.4 evaluates supernode clustering on both analogy and multi-hop graphs. Section 4.5 validates summary graphs with steering interventions.

The main quantitative results support the three claims. Adding upstream relevance to the pruning score preserves graph completeness and replacement score at matched graph sizes. The ILP clusterer achieves the strongest structural preservation among the clustering baselines: at the $p65$ role-cosine threshold it obtains internalized edge mass of 0.048 on both analogy and multi-hop graphs, with DAG loss of 0.056 on analogies and 0.038 on multi-hop graphs. Finally, entity-swap steering shows that Output-role supernodes contain transferable answer information, with top-1 hit rates up to 0.4400 and top-5 hit rates up to 0.6800 in the tested relations.

4.2 Common Experimental Setup

4.2.1 Datasets

Synthetic analogies from BATS. The main quantitative evaluation uses synthetic analogy prompts built from the Bigger Analogy Test Set (BATS). BATS is a benchmark for probing relational and semantic knowledge in word embeddings and language models. We use entities from its lexicography category and construct prompts of the form “The saying goes: A is to B as C is to D ”, where A , B , C , and D come from the same subcategory. The model must predict D from the relation expressed by $A \rightarrow B$ and the query entity C .

The dataset contains 100 prompts in total, with 10 prompts for each of the

following relation types: country–capital, country–language, UK city–county, name–nationality, name–occupation, animal–young, animal–sound, animal–shelter, object–color, and male–female. The analogy setting is useful because the answer entity D should be recoverable by applying the relation between A and B to C . A good summary should therefore preserve features that carry the source entities, the relation, the query entity, and the predicted answer. For the quantitative pruning and clustering experiments, we keep prompts that the model answers correctly with confidence above 20%. This filter ensures that every attribution graph explains a prediction the model actually makes.

Multi-hop reasoning. The second dataset follows the multi-hop queries of Ameisen et al. [1]. Each prompt requires several reasoning steps before reaching the answer, so it tests whether the summary pipeline can preserve causal structure beyond a single local association. The dataset contains 36 prompts covering basic knowledge in geography, biology, physics, and common knowledge. These graphs are used to test whether the clustering results remain stable beyond the BATS-derived analogy graphs.

4.2.2 Environment and implementation

Experiments run on a single workstation with an NVIDIA GeForce RTX 4090 GPU (24 GB VRAM) and 128 GB of RAM, under Ubuntu 22.04 LTS. The implementation uses Python 3.13.5. Attribution graphs are generated with the circuit-tracing library [15]. The summary graphs reuse the same feature metadata and are rendered with an adapted version of the library’s user interface.

4.2.3 Default pipeline configuration

The model and transcoder set used in the circuit tracing backbone to generate the attribution graph are google/gemma-2-2b [16] and mntss/clt-gemma-2-2b-426k [17].

Unless stated otherwise, pruning uses the influence–relevance importance score from Chapter 3 with an edge retained-mass threshold of 0.95. The pruning sweep varies the node retained-mass threshold and the mixture parameter α in Eq. (3.7). The clustering experiments use pruned analogy graphs from the $\alpha = 0.5$ condition, which balances downstream influence and upstream relevance. The exact clusterer is the binary integer linear program (ILP) of Eq. (3.17), solved through SciPy’s `milp` interface to the HiGHS mixed-integer solver with an explicit time limit. Thus the mathematical model is an ILP, while the software backend is a mixed-integer solver.

4.3 Experiment 1: Pruning Faithfulness

4.3.1 Question and comparison

The pruning experiment asks whether the proposed influence–relevance score can remove graph mass while preserving the end-to-end computation represented by the attribution graph. The direct baseline is influence-only pruning from Ameisen et al. [1]. It sets $\alpha = 1$ in Eq. (3.7), so a node is ranked only by downstream reach to the target logit. The proposed setting uses smaller values of α to combine downstream influence with upstream relevance to input tokens. This comparison tests whether relevance improves the semantic focus of the retained graph without sacrificing graph-level faithfulness.

4.3.2 Metrics

Pruning is evaluated with two graph-faithfulness metrics. The first metric tests whether influence can still be traced from the output back to retained input tokens. The second metric tests how completely retained nodes explain incoming edge mass throughout the graph.

Replacement score. Replacement score measures the fraction of end-to-end influence from input tokens to output logits that flows through the retained part of the graph. Let V' and E' be the retained node and edge sets from the pruning stage. The complement $V \setminus V'$ denotes attribution-graph nodes outside the pruned graph. Let W' be the retained adjacency from Chapter 3, with all unretained nodes and edges zeroed, and let Inf'_v be the influence score computed from W' using Eq. (3.3). The score is

$$\text{Rep} = \frac{\sum_{v \in V' \cap V_{\text{emb}}} \text{Inf}'_v}{\sum_{v \in V} \text{Inf}'_v}. \quad (4.1)$$

This is a strict score: it is high only when output influence can be traced all the way back to input tokens through retained pathways, rather than being absorbed by nodes outside the pruned graph.

Graph completeness. Graph completeness measures the fraction of incoming edges to all nodes that originate from the pruned graph, weighted by each node’s influence on the output. As in Chapter 3, $(W')^{\text{in}}$ is column-normalized with zero columns left at zero when a node has no incoming edge mass. Let

$$q'_v = \begin{cases} \sum_{u \in V'} (W')^{\text{in}}_{uv} = 1 - \sum_{u \in V \setminus V'} (W')^{\text{in}}_{uv}, & \text{if node } v \text{ has positive incoming edge mass in } W' \\ 0, & \text{otherwise.} \end{cases}$$

This is the fraction of node v ’s normalized incoming edge mass that comes from retained nodes. The score is

$$\text{Comp} = \frac{\sum_{v \in V} q'_v(\text{Inf}'_v)}{\sum_{v \in V} (\text{Inf}'_v)}. \quad (4.2)$$

Unlike replacement score, completeness gives partial credit to nodes that are mostly explained by retained features even when some influence remains assigned to nodes outside the pruned graph.

Irrelevant feature count The irrelevant feature count measures the number of features that are not relevant to the information we care about. Ameisen et al. [1] mention the “boring circuit” problem where a large number of active features and graph edges are responsible for basic, obvious roles, such as task-related features, punctuation, stopwords, and format scaffolding. We usually care more about how the model derives the semantics required to output the next token than about syntactic features. Therefore, this metric is used to measure the proportion of features that are not relevant to the information we care about. For the analogies dataset, we label the token positions of the entities A , B , C , and the final “to” token as relevant, and the remaining positions as irrelevant. After pruning, we report two counts of retained nodes at irrelevant positions: the number of cross-layer transcoder feature nodes and the number of input token (embedding) nodes. Lower values mean that pruning discards more of the syntactic scaffolding. We prune all attribution graphs with a node retained-mass threshold of 0.02 and compare settings.

4.3.3 Results

Table 4.1 reports the pruning results averaged over the 100 analogy graphs. Influence-only pruning attains the highest graph completeness (0.604) and replacement score (0.437), but it retains the most irrelevant nodes: on average 2.29 feature nodes and 9.00 input-token nodes lie on irrelevant positions. Adding upstream relevance ($\alpha < 1$) removes almost all of this scaffolding while losing at most 0.02 completeness and 0.02 replacement score. The two normalizations trade off the feature and token axes differently. Entmax, which assigns exactly zero weight to low-importance tokens, drives the irrelevant token count to about 0.1, effectively dropping every irrelevant input node; softmax keeps a denser token weighting and therefore retains 2.8–3.5 irrelevant token nodes. On the feature axis both normalizations are comparable, with fewer than 0.13 irrelevant feature nodes. These results support the claim that influence–relevance pruning preserves graph faithfulness at matched thresholds while discarding features and tokens that are not semantically relevant to the analogy.

For the rest of the experiments, we use our settings with Entmax normalization and $\alpha = 0.5$.

Table 4.1: Pruning results on 100 BATS analogy graphs at edge retained-mass threshold 0.95 and node retained-mass threshold 0.02. Values are means across graphs. Irrelevant feature and token counts are the numbers of retained feature and input-token nodes at irrelevant positions (lower is better); graph completeness and replacement score are the graph-faithfulness metrics of Eq. (4.2) and Eq. (4.1) (higher is better). The best value in each column is in bold and the second-best value is underlined; the $\alpha = 1.0$ baseline ignores token weights, so its normalization is not applicable.

Method	Normalization	α	Irrelevant feature count ↓	Irrelevant token count ↓	Graph completeness ↑	Replacement score ↑
Influence only	–	1.0	2.29	9.00	0.604	0.437
Ours	Entmax 1.25	0.75	0.13	<u>0.12</u>	<u>0.598</u>	<u>0.425</u>
Ours	Entmax 1.25	0.5	0.12	0.11	0.589	0.421
Ours	Softmax	0.75	0.04	3.48	0.591	0.423
Ours	Softmax	0.5	<u>0.07</u>	2.77	0.583	0.418

4.4 Experiment 2: Causal Supernode Clustering

4.4.1 Question and setup

The clustering experiment asks whether the proposed ILP clusterer forms supernodes with coherent causal roles. The experiment uses 100 pruned analogy graphs from the entmax $\alpha = 0.5$ condition. For the ILP solver, we set the role-cosine threshold to the 65th percentile of allowed feature pairs, denoted p_{65} , and set the upper bound for the number of supernodes to $K_{\max} = 20$. All clustering methods are matched to the same number of feature supernodes as the proposed summary on each graph, so the comparison isolates partition quality rather than graph size. Let $F = V'_{\text{feature}}$ be the retained feature nodes, let f be the partition from clustering, and let $r(i)$ be the role vector of Eq. (3.11). All edge-mass formulas use the retained edge set E' and the retained adjacency W' , with the convention that W'_{uv} is the edge from source u to target v .

4.4.2 Baselines

Spectral clustering on role vectors. Spectral clustering on role vectors uses the same role-vector representation as the proposed clusterer, but replaces the ILP objective with a standard spectral partitioning algorithm. The affinity is a rectified cosine similarity between role vectors. This baseline tests whether a generic graph-free clustering method can recover comparable supernodes from the same feature representation.

K-means on role vectors. K-means on role vectors first unit-normalizes each role vector and then minimizes Euclidean within-cluster variance. After normalization, Euclidean distance is a cosine-distance surrogate, so this is a strong geometric compactness baseline. It tests whether tight role-vector clusters are sufficient for

causal summary quality.

Spectral clustering on adjacency. Spectral clustering on adjacency ignores the role vectors and instead uses the symmetrized absolute pruned adjacency matrix as its affinity. It groups features that are strongly connected in the retained graph. This baseline tests whether connectivity alone is enough, without the incoming and outgoing role profiles used by the proposed method.

Random same-size clusters. The random same-size baseline keeps the same feature-cluster size distribution as the proposed summary but randomly assigns features to clusters. It controls for the effect of cluster count and cluster sizes. Any method that does not beat this baseline is not extracting causal or geometric structure from the graph.

4.4.3 Metrics

All clustering metrics are computed independently for each graph and then averaged as unweighted graph means. Same-cluster feature pairs are always used exactly; cross-cluster pairs are sampled with a fixed seed only when their count exceeds the evaluation cap.

Role gap. Role gap measures whether full signed causal-role vectors are more similar inside a supernode than across different supernodes. Let $r_i = [v_i^{\text{out}}; v_i^{\text{in}}]$ be the signed role vector from Eq. (3.11), and let $\mathcal{Q}_{\text{same}} = \{(i, j) : i, j \in F, i < j, f(i) = f(j)\}$ and $\mathcal{Q}_{\text{diff}} = \{(i, j) : i, j \in F, i < j, f(i) \neq f(j)\}$ be same-cluster and cross-cluster feature pairs. The cosine uses $\cos(x, y) = x^\top y / (\|x\| \|y\| + \epsilon)$ with $\epsilon = 10^{-12}$. The score is

$$\text{RoleGap} = \overline{\cos(r_i, r_j)}_{(i,j) \in \mathcal{Q}_{\text{same}}} - \overline{\cos(r_i, r_j)}_{(i,j) \in \mathcal{Q}_{\text{diff}}}. \quad (4.3)$$

Higher role gap means that members of the same supernode have more similar combined incoming and outgoing signed roles than features assigned to different supernodes.

Internalized edge mass. Internalized edge mass measures how much fine-grained causal weight disappears inside feature supernodes. It uses the same retained-edge denominator as the causal-information constraint in Chapter 3:

$$\text{Internalized} = \frac{\sum_{(u,v) \in E' : u,v \in F, f(u)=f(v)} |W'_{uv}|}{\sum_{(u,v) \in E'} |W'_{uv}|}. \quad (4.4)$$

Lower internalized edge mass means that more retained causal structure remains visible as edges between supernodes rather than being hidden inside feature supernodes.

DAG loss. DAG loss measures the fraction of aggregated superedge mass that points backward under the summary graph’s topological order. Let $A_{C \rightarrow C'}$ be the raw block-sum edge from supernode C to supernode C' before antiparallel merging or backward-edge removal, and let $\sigma(C)$ be the final supernode order. The backward set is $R_\sigma = \{(C, C') : A_{C \rightarrow C'} \neq 0, \sigma(C) > \sigma(C')\}$. The score is

$$\text{DAGLoss} = \frac{\sum_{(C, C') \in R_\sigma} |A_{C \rightarrow C'}|}{\sum_{C \neq C'} |A_{C \rightarrow C'}| + \epsilon}. \quad (4.5)$$

Lower DAG loss means the partition is more compatible with the final acyclic summary graph.

4.4.4 Results

Table 4.2 reports method means on entmax-pruned analogy and multihop graphs. K-means obtains the largest role gap on both datasets, which shows that direct geometric clustering can form compact role-vector groups. However, those groups hide more feature–feature edge mass and require more backward-edge removal than the proposed ILP summary when converted into an acyclic summary graph.

The gain columns compare each method with the random same-size baseline. Positive values are better. Since the random-baseline role gap is near zero, the role-gain column reports the absolute difference $m - m_{\text{rand}}$ rather than a percentage. For internalized mass and DAG loss, gain is the percentage reduction from the random baseline, $100(m_{\text{rand}} - m)/m_{\text{rand}}$.

Table 4.2: Clustering evaluation on entmax-pruned analogy and multihop graphs. Higher role gap indicates more coherent signed causal roles inside supernodes. Lower internalized edge mass and lower DAG loss indicate better preservation of visible causal structure. Gains are measured against the random same-size baseline. **Best** and second-best values are highlighted for each metric.

Method	Analogies (100)						Multihop (36)					
	Role gap $\uparrow$	Int. mass $\downarrow$	DAG loss $\downarrow$	Role $\Delta \uparrow$	Int. gain $\uparrow$	DAG gain $\uparrow$	Role gap $\uparrow$	Int. mass $\downarrow$	DAG loss $\downarrow$	Role $\Delta \uparrow$	Int. gain $\uparrow$	DAG gain $\uparrow$
Random same-size clusters	-0.012	<u>0.082</u>	0.124	+0.000	<u>+0.0%</u>	+0.0%	0.004	<u>0.062</u>	0.091	+0.000	<u>+0.0%</u>	+0.0%
Spectral on adjacency	0.108	0.187	0.114	+0.121	-128.2%	+8.1%	0.081	0.118	0.076	+0.077	-89.7%	+16.0%
Spectral on role vectors	0.562	0.097	<u>0.078</u>	+0.575	-18.1%	<u>+37.4%</u>	0.479	0.087	<u>0.051</u>	+0.474	-40.0%	<u>+43.6%</u>
K-means on role vectors	0.713	0.084	0.092	+0.726	-2.3%	+25.9%	0.612	0.068	0.058	+0.608	-9.1%	+36.0%
Ours ILP	<u>0.675</u>	0.048	0.056	<u>+0.688</u>	+41.1%	+54.9%	<u>0.591</u>	0.048	0.038	<u>+0.586</u>	+22.7%	+58.5%

The edge-mass metrics identify the intended advantage of the ILP formulation. The ILP row has the lowest internalized edge mass and DAG loss on both datasets. Relative to random same-size clusters, ILP reduces internalized mass by 41.1% on analogies and 22.7% on multihop, and reduces DAG loss by 54.9% on analogies

and 58.5% on multihop. K-means gives the largest role-gap improvement, but it increases internalized mass over the random baseline on both datasets. The ILP therefore sacrifices some role-vector compactness to preserve more visible causal structure in the final summary graph.

4.5 Experiment 3: Intervention Validation via Entity-swap

4.5.1 Question and setup

The intervention experiment asks whether the summary graph identifies supernodes that are not only interpretable after inspection, but also causally usable. In the analogy task, an Output-role supernode should contain features that promote the answer entity D for a prompt of the form “ A is to B as C is to”. If this interpretation is correct, replacing the source prompt’s Output-role features with the corresponding features from a donor prompt should move the model toward the donor answer.

The experiment uses ordered source–donor pairs from the BATS analogy dataset. The source and donor prompts must come from the same relation type, so the intervention changes the answer entity while keeping the relation fixed. Pairs are skipped when the source and donor have the same target token or when either summary graph lacks usable Output-role CLT features. This filtering keeps the test focused on answer transfer rather than on degenerate pairs or missing summary nodes.

4.5.2 Intervention protocol

Entity-swap steering evaluates whether the Output supernodes discovered by the summary graph are causally meaningful. For each analogy relation, we pair a source prompt with a donor prompt from the same relation. We suppress the source Output supernode and inject the donor Output supernode at the corresponding position. If the summary graph has identified an answer-carrying supernode, this intervention should move the model prediction away from the source answer and toward the donor answer.

Figure 4.1 shows the summary graph for one animal–young prompt and the corresponding intervention view after replacing features that promote “kid” with features that promote “calf”.

4.5.3 Metrics

The primary metric is top-1 hit rate. A pair is counted as a top-1 hit only when the steered model’s most likely next token is the donor target token. This is a strict success criterion because the intervention must overcome the clean source answer and make the donor answer the model’s final prediction.

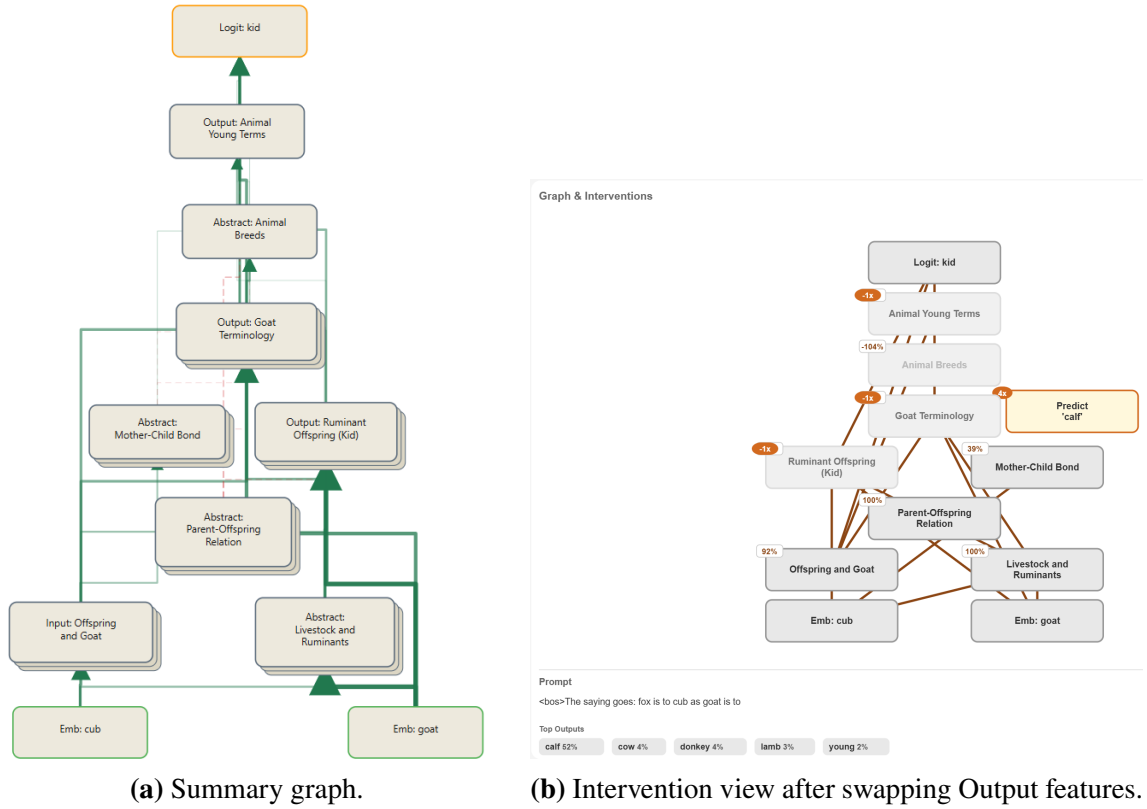


Figure 4.1: Summary graph and intervention view for the prompt “The saying goes: fox is to cub as goat is to”. The model output changes after swapping Output features that promote “kid” with features that promote “calf”.

The secondary metrics measure weaker forms of transfer. Top-5 hit rate counts whether the donor target appears among the five highest-probability next tokens. The donor probability change Δp_{donor} is the steered donor target probability minus its clean value, and the source probability change Δp_{source} is defined analogously for the source target. These metrics are useful because a summary supernode can increase the donor answer evidence and suppress the source answer without always making the donor answer the top-1 prediction.

4.5.4 Results

Table 4.3 reports entity-swap transfer for two BATS relations with the source suppression factor fixed at -2 . Each row aggregates 50 attempted source–donor pairs for each donor factor, and hit rates are computed over all attempted pairs. The results show that Output-role supernodes carry transferable answer information: across the four settings, top-1 hit rates range from 0.1600 to 0.4400, and top-5 hit rates range from 0.2800 to 0.6800.

The transfer rate depends strongly on the relation. For capital–country pairs, the best setting obtains a top-1 hit rate of 0.2000 and a top-5 hit rate of 0.4000. For person–nationality pairs, the best setting obtains a top-1 hit rate of 0.4400 and a

Table 4.3: Entity-swap supernode steering on 50 BATS analogy pairs per relation with the source factor fixed at -2 . Donor Output features are injected with the grouped donor factor. Hit rates are over all attempted pairs, and Δp values are steered probabilities minus clean probabilities. The best value for each relation and metric is shown in bold.

Relation	Donor factor 4				Donor factor 8			
	Top-1 hit	Top-5 hit	$\Delta p_{\text{donor}} \uparrow$	$\Delta p_{\text{source}} \downarrow$	Top-1 hit	Top-5 hit	$\Delta p_{\text{donor}} \uparrow$	$\Delta p_{\text{source}} \downarrow$
capital–country	0.1600	0.2800	0.0944	-0.5741	0.2000	0.4000	0.0817	-0.6746
person–nationality	0.4400	0.6800	0.2188	-0.5328	0.3600	0.6000	0.1617	-0.5747

top-5 hit rate of 0.6800. The donor probability change shows the same pattern: the largest Δp_{donor} for capital–country is 0.0944, whereas the largest value for person–nationality is 0.2188.

The factor sweep also shows that stronger donor injection is not uniformly better. Increasing the donor factor from 4 to 8 improves the capital–country top-1 and top-5 hit rates, but it reduces the person–nationality hit rates and Δp_{donor} . This behavior suggests that the entity-swap intervention is sensitive to activation scale, so the results should be interpreted as evidence for transferable Output-role information rather than as an optimized steering method. Overall, the intervention validates the summary graph at the supernode level: the same nodes that receive answer-role labels can be used to move the model toward a different answer entity.

4.6 Chapter Summary

This chapter evaluated the summarization pipeline through three experiments. The pruning experiment showed that adding upstream relevance to the importance score does not substantially reduce replacement score or graph completeness at matched graph sizes, while discarding features that are not semantically relevant to the input prompt. The clustering experiment showed that the ILP clusterer yields the lowest internalized edge mass and DAG loss among the tested baselines, while K-means yields the strongest role-vector gaps. The end-to-end steering experiment validated the final summary graph’s ability to predict the behavior of the model under intervention. Taken together, these results indicate that the proposed pipeline preserves important causal structure while compressing attribution graphs into more interpretable summary graphs. Chapter 6 discusses the remaining limitations, and future work.

CHAPTER 5. CONCLUSIONS

5.1 Summary

This thesis addressed the problem of making attribution graphs from circuit tracing more readable without discarding their causal content. Attribution graphs provide a prompt-specific account of how a language model produces a prediction, but they are often too large and dense for direct inspection. To address this limitation, the thesis formalized the problem of attribution-graph summarization and proposed an automated method for producing compact, human-readable summary graphs.

The proposed method consists of three stages: pruning, clustering, and visualizing. The pruning stage removes nodes and edges that are unlikely to participate in the selected input-to-output computation. Instead of using only downstream influence on the target logit, it combines downstream influence with upstream relevance from the selected input tokens, so that retained nodes must be both reached from the prompt context and relevant to the prediction. The clustering stage groups retained feature nodes into supernodes. It formulates clustering as an optimization problem that rewards atomicity, so that features inside a supernode have similar signed causal roles, while also preserving visible causal structure between supernodes. The visualizing stage aggregates the clustered graph into a directed acyclic summary graph and assigns labels and roles to the resulting supernodes. The final output is therefore a smaller graph that presents the model’s computation as a sequence of interpretable causal steps.

The empirical results provide evidence that the summary graphs satisfy the main desiderata proposed in this thesis. The pruning experiments show that adding upstream relevance to the importance score preserves replacement score and graph completeness at matched graph sizes, while filtering out features that are less semantically connected to the selected prompt tokens. The clustering experiments show that the proposed ILP clusterer gives stronger causal-structure preservation than the tested clustering baselines, as measured by lower internalized edge mass and lower DAG loss, while still maintaining coherent role-vector clusters. The intervention experiments further validate the final summaries. Entity-swap steering shows that Output supernodes often contain answer-specific information that can be suppressed or transferred across prompts. Together, these results suggest that the proposed pipeline compresses attribution graphs while preserving important parts of their causal structure.

The approach nevertheless has several limitations. First, the relevance score

uses SHAP values as a proxy for input-token importance. Although this helps suppress uninformative prompt tokens in practice, SHAP is a non-linear attribution method, and it is not clear how it transfers to linear attribution graphs. The thesis therefore uses SHAP to approximate human rationales rather than as an evaluation target. Second, the experiments are limited in scale. They focus mainly on analogy and multi-hop reasoning prompts for one circuit-tracing backbone, and do not test longer prompts, larger models, or a broader range of behaviors. Third, there is no established end-to-end baseline for summarizing CLT attribution graphs. The thesis therefore compares against stage-level baselines such as influence-only pruning and classical clustering methods, but it cannot yet compare against a mature competing summarization pipeline. Finally, the method inherits limitations from the underlying replacement model: if important behavior is represented in reconstruction error, attention dynamics, or features not captured well by the transcoder, the summary graph may omit or distort part of the original model computation.

Despite these limitations, the thesis provides a concrete step toward scalable mechanistic analysis of language models. It replaces manual pruning and manual grouping with explicit objectives, produces summary graphs that can be inspected directly, and validates those summaries using causal interventions. This makes attribution-graph analysis more systematic and opens the possibility of applying circuit-tracing tools to larger collections of prompts. In the long term, methods of this kind could support the study of model internals, the diagnosis of unexpected model behavior, and the investigation of failure modes such as hallucination, jailbreak susceptibility, and unfaithful reasoning traces.

5.2 Future Work

Future work should strengthen both the summarization method and its evaluation. A first direction is to replace the current SHAP-based relevance proxy with a relevance measure that aligns better with the attribution graph. Such a measure would express upstream and downstream importance within the same linear framework, while approximating human rationales closely.

A second direction is to evaluate the pipeline on a broader set of models, prompts, and tasks. The present experiments focus on controlled analogy and multi-hop reasoning settings. Future evaluations should include longer prompts, tasks with multiple competing answers, and models with different sizes or replacement-model quality. This would clarify which parts of the pipeline are robust and which depend on the structure of the current experimental setting.

A third direction is to develop stronger end-to-end baselines. Since no standard

method currently exists for CLT attribution-graph summarization, future work could compare against expert-written summaries, adapted neuron-circuit clustering methods, ADAG-style attribution-profile clustering, or ablations of the proposed pipeline. Such comparisons would make it easier to separate the contribution of each stage from the contribution of the complete pipeline.

Finally, future work should use the proposed pipeline to study the biology of language models, investigate failure modes, and build more reliable evidence about model behavior.

SHORT NOTICES ON REFERENCE

All sources cited in this thesis are formally published works — peer-reviewed papers, conference proceedings, and technical reports issued by their authoring organizations. Each reference is numbered in order of first appearance and cited in the text by its number in square brackets, for example [1]. The full bibliographic details of every cited work are listed in the REFERENCE section that follows.

REFERENCE

- [1] Emmanuel Ameisen **and others**. “Circuit Tracing: Revealing Computational Graphs in Language Models”. **in** *Transformer Circuits Thread*: (2025). URL: <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- [2] Aryaman Arora **and others**. “Language Model Circuits Are Sparse in the Neuron Basis”. **in** *arXiv preprint arXiv:2601.22594*: (2026). URL: <https://arxiv.org/abs/2601.22594>.
- [3] Aryaman Arora **and others**. “ADAG: Automatically Describing Attribution Graphs”. **in** *arXiv preprint arXiv:2604.07615*: (2026). URL: <https://arxiv.org/abs/2604.07615>.
- [4] Ashish Vaswani **and others**. “Attention is All you Need”. **in** *Advances in Neural Information Processing Systems (NeurIPS)*: 2017.
- [5] Nelson Elhage **and others**. “A Mathematical Framework for Transformer Circuits”. **in** *Transformer Circuits Thread*: (2021). URL: <https://transformer-circuits.pub/2021/framework/index.html>.
- [6] Chris Olah **and others**. “Zoom In: An Introduction to Circuits”. **in** *Distill*: (2020). DOI: 10.23915/distill.00024.001.
- [7] Nelson Elhage **and others**. “Toy Models of Superposition”. **in** *Transformer Circuits Thread*: (2022). URL: https://transformer-circuits.pub/2022/toy_model/index.html.
- [8] Hoagy Cunningham **and others**. “Sparse Autoencoders Find Highly Interpretable Features in Language Models”. **in** *arXiv preprint arXiv:2309.08600*: (2023).
- [9] David Chanin **and others**. “A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders”. **in** *arXiv preprint arXiv:2409.14507*: (2024). Accepted at NeurIPS 2025 (Oral). DOI: 10.48550/arXiv.2409.14507.
- [10] Jacob Dunefsky, Philippe Chlenski **and** Neel Nanda. “Transcoders Find Interpretable LLM Feature Circuits”. **in** *arXiv preprint arXiv:2406.11944*: (2024).
- [11] Jack Lindsey **and others**. “On the Biology of a Large Language Model”. **in** *Transformer Circuits Thread*: (2025). URL: <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- [12] Steven Bills **and others**. “Language Models can Explain Neurons in Language Models”. **in** *OpenAI*: (2023). URL: <https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html>.

-
- [13] Anna Zeng **and others**. “Causal DAG Summarization”. in *Proceedings of the VLDB Endowment*: 18.6 (2025), pages 1933–1947. DOI: 10.14778/3725688.3725717.
- [14] Nelson Elhage **and others**. “Softmax Linear Units”. in *Transformer Circuits Thread*: (2022). URL: <https://transformer-circuits.pub/2022/solu/index.html>.
- [15] Michael Hanna **and others**. *circuit-tracer*. <https://github.com/decoderresearch/circuit-tracer>. The first two authors contributed equally and are listed alphabetically. 2025.
- [16] Google. *Gemma 2 2B*. <https://huggingface.co/google/gemma-2-2b>. Hugging Face model repository. Accessed: 2026-07-06. 2024.
- [17] mntss. *clt-gemma-2-2b-426k*. <https://huggingface.co/mntss/clt-gemma-2-2b-426k>. Hugging Face model repository. Accessed: 2026-07-06. 2025.

APPENDIX

A. APPENDIX

A.1 SHAP Value Calculation

The token weight vector τ in Section 3.2 is initialized from per-token SHAP attributions computed on the same causal language model that produced the attribution graph. The implementation follows the high-level SHAP language-model interface: a `Text` masker perturbs input token segments, and `TeacherForcing` scores each masked prompt by the log-odds of producing a fixed continuation Y .

Explained target. By default, Y is the one-token continuation sampled from `model.generate` on the unmasked prompt. When the graph specifies a target logit, Y is the single vocabulary token decoded from that logit id, so the attribution aligns with the graph’s prediction target rather than a fresh sample.

Masking. Input segments are masked with the placeholder `. . .` and collapsed into one mask token (`collapse_mask_token=True`). A leading BOS token is removed from the SHAP input when present, then zero-padded back into the final weight vector so lengths match `prompt_tokens`. For chat-formatted prompts, `pin_special_tokens=True` replaces the default masker with a variant that marks tokenizer special and added tokens as invariant, so scaffold tokens such as `<|im_start|>` are never perturbed. Optionally, `masker_keep_prefix=k` leaves the first k segments fixed during masking.

From raw SHAP values to τ . The explainer returns a matrix of shape (input segments $\times$ output tokens). Contributions are summed over the output axis (with a transpose check when the backend returns segment counts along the wrong axis) to obtain one scalar per input segment. Special tokens matching `<. . .>` and any prefix-pinned positions receive zero mass after normalization.

The raw signed scores are passed directly to the selected normalizer; the implementation does not take a positive part or absolute value before normalization. These scores are mapped to a probability vector over non-special prompt tokens via one of four normalizers: softmax, sparsemax, entmax-1.5, or entmax with $\alpha \in (1, 2]$ (default $\alpha = 1.25$). Special-token positions are masked before normalization and forced to zero afterward; remaining mass is renormalized to sum to one. The graph entry point `get_token_attribution_from_graph` reads `prompt`, `prompt_tokens`, and the target-logit feature id from graph metadata, then returns this normalized vector for use as τ .